



Building

Describe the appropriate approaches to building test data strategy and unit test to ensure successful code (positive, negative, permission-based, large data volume).

Develop your career with Salesforce Training and Certification Preparation

Table of Contents

-  [Test Data Strategies](#)
-  [Unit Testing Strategies](#)
-  [Business Scenario](#)
-  [Requirements & Solutions](#)



After studying this topic, you should be able to:

-  Explain the importance of creating test data for unit tests in Salesforce.
-  Identify the various test data creation techniques.
-  Describe the various strategies available for unit testing in Salesforce.
-  Recommend appropriate test data and unit testing strategies for various technical requirements.



Introduction

In Salesforce, **creating test data** for Apex tests is necessary to ensure that tests **do not depend on production data** and are **repeatable**. Test data can be created using **brute force**, a **test setup method**, a **test data factory**, or a **CSV data file** uploaded as a **static resource**.

Various unit testing strategies are available for Apex unit tests. **Positive tests** are used to return expected results when a valid input is provided. **Negative tests** are used to ensure that code handles invalid data and unexpected user input correctly. **Permission-based tests** ensure that code behaves as expected when it is run as a user with or without specific permissions. **Mocks and stubs** facilitate unit testing by simulating real objects and methods, allowing developers to test code in isolation from external dependencies.



Test Data & Unit Testing Strategies

Test Data Creation Methods

Test data can be created through brute force, a test setup method, test data factory, and a CSV file uploaded as a static resource.



Positive Tests

Positive unit tests are tests that return expected results when valid input data is provided.



Negative Tests

Negative unit tests ensure that code handles invalid data and unexpected user input correctly.



Mocks and Stubs

Mocks and stubs facilitate unit testing by simulating real objects and methods, allowing developers to test code in isolation from external dependencies.



Permission-Based Tests

Permission-based tests ensure that code behaves as expected when it is run as a user with or without specific permissions.



Test Data Strategies

Table of Contents



Test Data

Creating isolated unit test data ensures that tests **do not depend** on production data and are **repeatable**.

❖ IMPORTANCE

Creating test data for unit tests ensures that tests are **consistent across runs** and **not dependent on production data**.

❖ DATA DEPENDENCY

Dependency on **production data** can lead to **deployment issues** due to **data differences** across environments.

❖ TRANSIENT BEHAVIOR

Data inserted by an Apex unit test **does not persist** after the test finishes execution, ensuring that **data cleanup is not required**.



Test Data

Creating isolated unit test data ensures that tests **do not depend** on production data and are **repeatable**.

❖ DEFAULT ACCESS

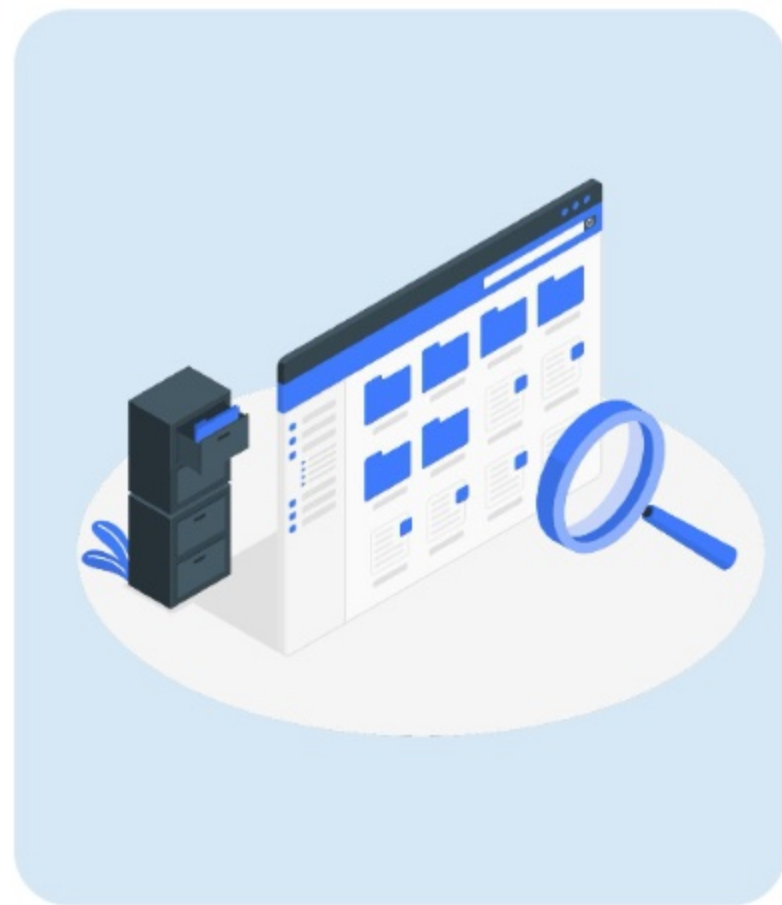
By default, Apex tests **do not have access** to pre-existing data in an org. Exceptions are **setup** and **metadata** objects, such as **User** and **Profile**.

❖ ORG DATA

A test class or method can be annotated with **IsTest(SeeAllData=true)** to open up data access to records in the org. However, the best practice is to run tests with **data silo** using the **IsTest(SeeAllData=false)** annotation.

❖ DATA QUERIES

The **IsTest(SeeAllData=true)** annotation applies to **data queries** only. New, updated, and deleted records are **rolled back** in Apex unit tests.



Creating Test Data

Several methods are available for **creating test data** for unit tests.

BRUTE FORCE

Records are **created and inserted manually** on each unit test. But this can lead to lengthy tests, especially when generating related records or when records contain a lot of field details.

TEST SETUP

A test setup method that creates test data can be created using the **@TestSetup annotation**. This method is called before every test method in the Apex class. The test data is reset between each and every test method's execution.



TEST DATA FACTORY

A **reusable Apex class** called Test Data Factory can be used to model the creation of records of one or more objects. A number of open-source data factories are available online.

CSV DATA FILE

A **CSV data file can be created and uploaded** as a **static resource**. Test data can be generated directly from the CSV file. This is useful when a **large data volume** is required for testing.

Test Setup Method

The following example shows the use of `@TestSetup` to define a method for creating test data.

```
@isTest
public class ServiceLevelAgreementTest {
    @TestSetup
    static void dataSetup() {
        Account a = new Account(Name='AccountDemo', Active__c='True', SLA_Status__c='Premium');
        insert a;
        Contract c = new Contract(AccountId=a.Id, Status='Draft', StartDate=Date.today(), ContractTerm=12);
        insert c;
    }
    @isTest
    static void testOpportunity() {
        Account a = [SELECT Id, SLA_Status__c FROM Account LIMIT 1];
        Contract c = [SELECT Id, ContractTerm FROM Contract LIMIT 1];
        Opportunity o = new Opportunity(Name='SLA', StageName='Closed Won', CloseDate=Date.today(), ContractId=c.Id, AccountId=a.Id);
        insert o;
        Opportunity r = [SELECT Id, CloseDate FROM Opportunity LIMIT 1];
        Date contractExpiry = r.CloseDate.addMonths(c.ContractTerm);
        System.assertEquals(r.Contract_Expiry__c, contractExpiry);
        System.assertEquals(r.Service_Status__c, a.SLA_Status__c);
    }
}
```

Test Data Creation

Test Data Retrieval

Result Verification

Test Data Factory

The following is an example of a Test Data Factory class that **creates** and **returns test accounts**.

```
1  /* The test utility class 'TestDataFactory' is used here to create and return
2   * account records based on the number of accounts required by test method. */
3
4
5  @isTest
6  public class TestDataFactory {
7      public static List<Account> createAccounts(Integer numAccounts) {
8          List<Account> accounts = new List<Account>();
9
10         for(Integer i=0;i<numAccounts;i++) {
11             Account acc = new Account(Name='Test Account ' + i);
12             accounts.add(acc);
13         }
14         insert accounts;
15
16         return accounts;
17     }
18 }
```


Using a Static Resource for Test Data

The following example shows a test method that **loads test data** from a **static resource**.

```
1  @isTest
2  ▼ public class StaticResourceTest {
3
4      @isTest
5  ▼  static void checkStaticFile() {
6          Test.loadData(Account.sObjectType, 'TestAccounts');
7          List<Account> accounts = [SELECT Name, Website, Phone FROM Account];
8          System.debug('accounts.size() = ' + accounts.size());
9  ▼  for (Account a : accounts) {
10             System.debug('account = ' + a);
11         }
12     }
13 }
```

To load test data, **sObject type** and **name of static resource** are specified.

Execution Log

Timestamp	Event	Details
23:36:32:241	USER_DEBUG	[8] DEBUG accounts.size() = 3
23:36:32:241	USER_DEBUG	[10] DEBUG account = Account:{Name=Rangers, Website=https://www.rangers.com, Phone=1234 5678, Id=0012v000035epFYAAY}
23:36:32:242	USER_DEBUG	[10] DEBUG account = Account:{Name=Trailblazers, Website=https://www.trailblazers.com, Phone=2345 6789, Id=0012v000035epFZAAY}
23:36:32:242	USER_DEBUG	[10] DEBUG account = Account:{Name=Campers, Website=https://www.campers.com, Phone=3456 7890, Id=0012v000035epFaAAI}

The **three records** from the **static resource** file are loaded.

Unit Testing Strategies

Table of Contents



Positive Tests

Positive unit tests are tests that return **expected results** when **valid input data** is provided.

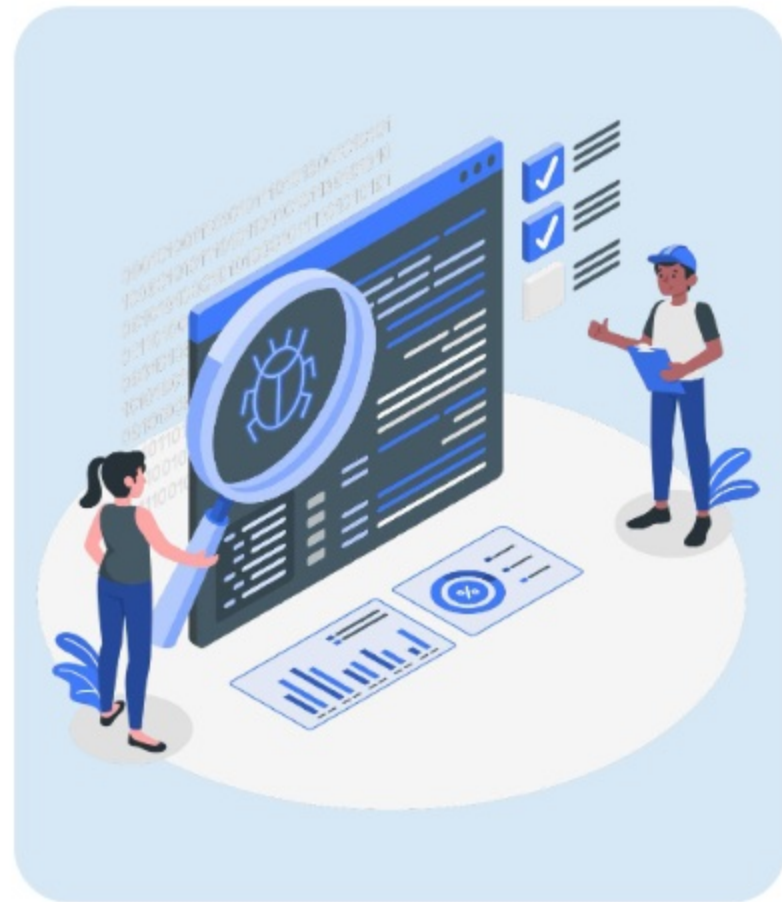
❖ IMPORTANCE

Positive testing ensures that **well-written code** performs as expected with **correct inputs**.

❖ PATTERN

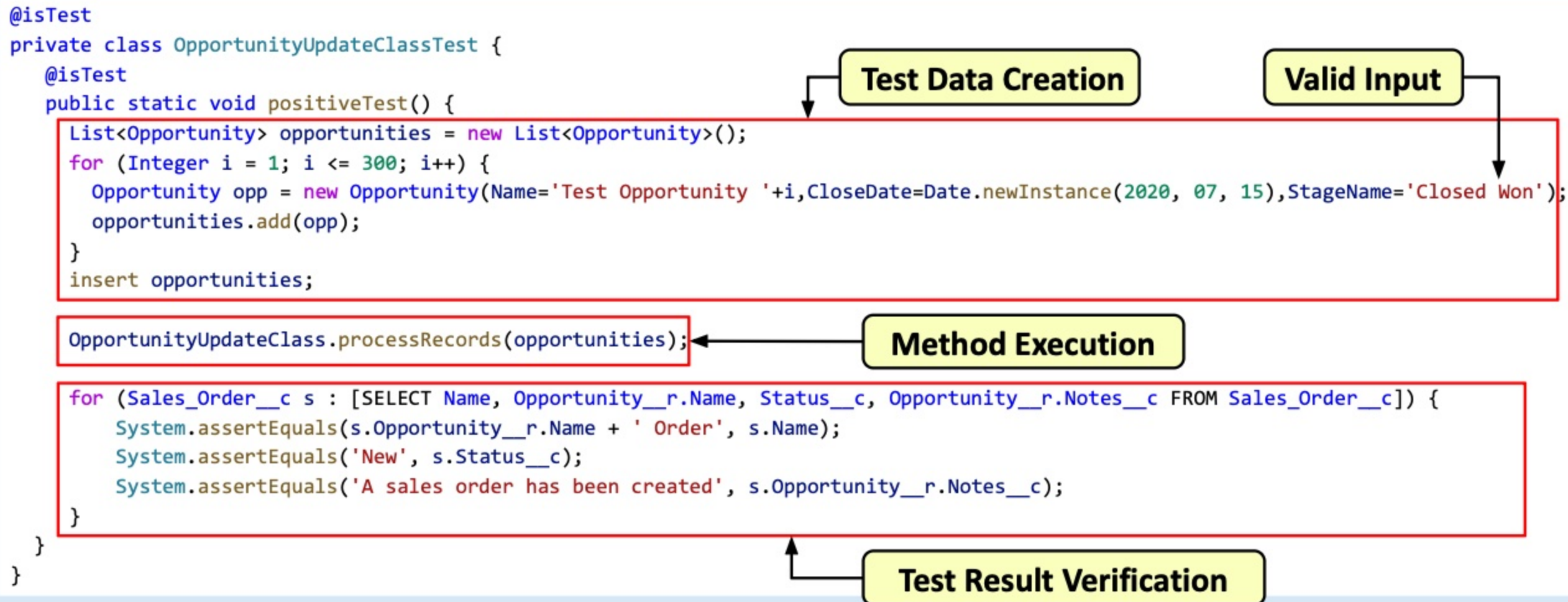
The pattern for writing positive unit tests uses the following steps:

- 1) Generate or load **test data**
- 2) Make assertions to **validate data**
- 3) Call **Test.startTest()**
- 4) **Execute** the code being tested
- 5) Call **Test.stopTest()**
- 6) Make assertions about **output**



Positive Tests

The following example shows the **positive test behavior** of a class when **valid input** is provided.



Negative Tests

Negative unit tests ensure that **code handles invalid data** and **unexpected user input correctly**.

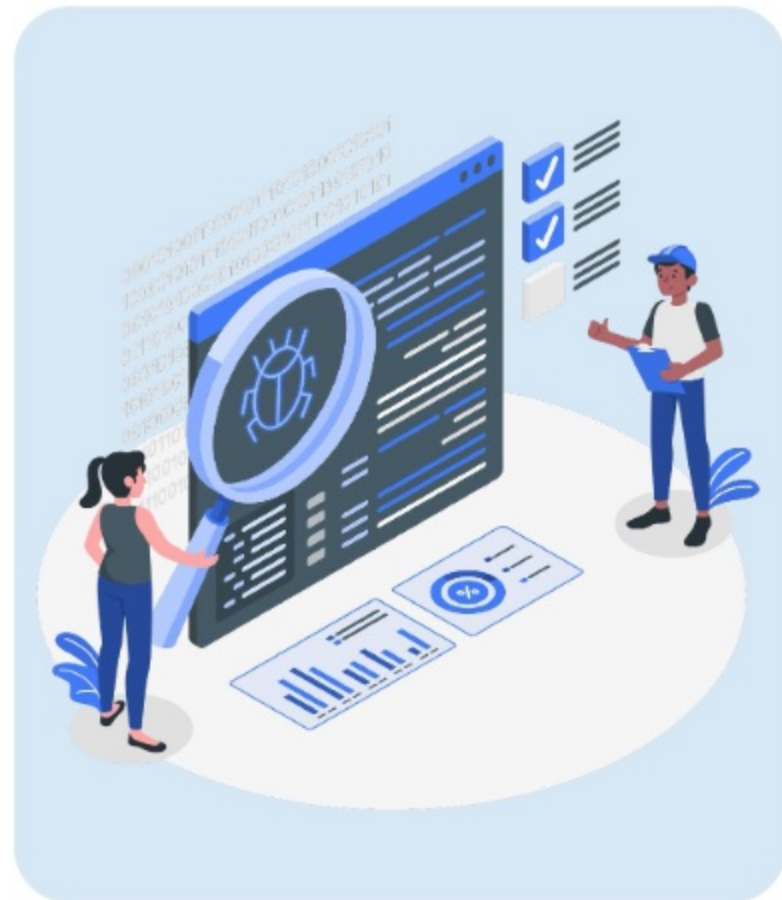
❖ IMPORTANCE

Negative testing ensures that the code is **fault-tolerant** and ensures that the code **gracefully handles exceptions, preventing failures** in batch processes or other operations.

❖ PATTERN

The pattern for writing positive unit tests uses the following steps:

- 1) Generate or load **test data**
- 2) Start a **try/catch block**
- 3) Call **Test.startTest()**
- 4) **Execute** the code that is expected to throw an exception
- 5) Call **Test.stopTest()**
- 6) Add an **assertion that fails** if no exception is thrown
- 7) **Catch** the expected exception



Negative Tests

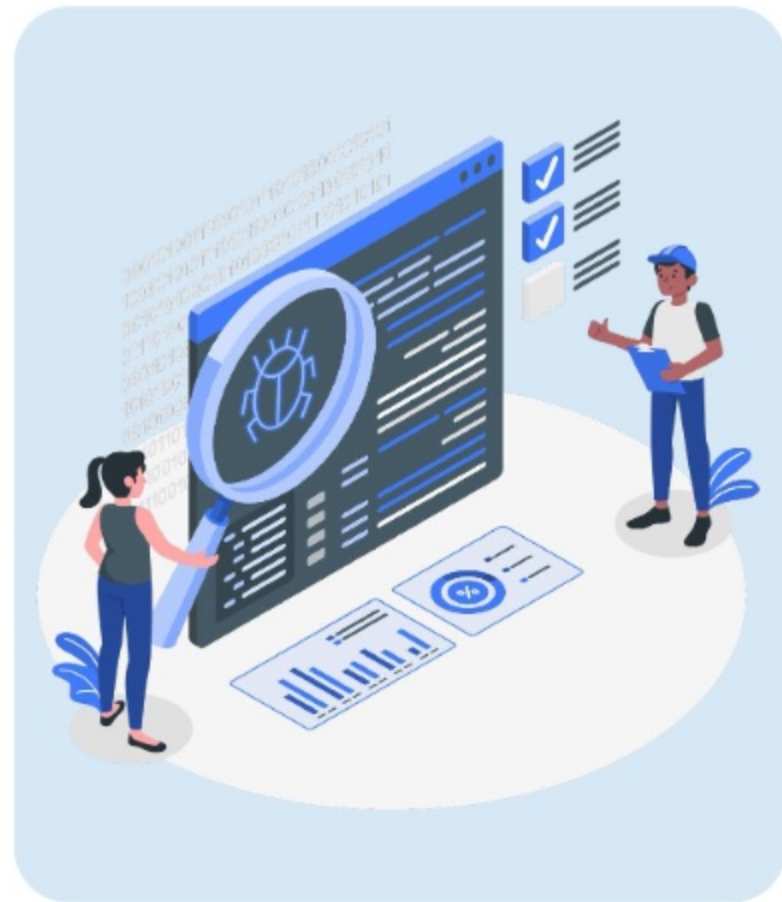
Negative unit tests ensure that **code handles invalid data** and **unexpected user input correctly**.

❖ BEST PRACTICES

Test data can be modified to **induce specific exceptions** and **validate error handling**. Both the **exception type** and **error message** should be checked in the catch block to **avoid false positives**.

❖ EXAMPLE

In a batch process, if the code **fails on the first record** in the batch, and it **does not gracefully handle exceptions**, the **remaining records are not processed**. Negative tests avoid such situations.



Negative Tests

The following example shows the **negative test behavior** of a class when **invalid input** is provided.

```
@isTest
private class OpportunityUpdateClassTest {
```

```
@isTest
public static void negativeTest() {
    List<Opportunity> opportunities = new List<Opportunity>();
    for (Integer i = 1; i <= 300; i++) {
        Opportunity opp = new Opportunity(Name='Test Opportunity '+i,CloseDate=Date.newInstance(2020, 07, 15),StageName='Closed
Lost');
        opportunities.add(opp);
    }
    insert opportunities;
```

```
    OpportunityUpdateClass.processRecords(opportunities);
```

```
    for (Opportunity o : [SELECT Id, Notes__c, (SELECT Id FROM Sales_Orders__r) FROM Opportunity WHERE Id IN :opportunities]) {
        System.assert(o.Sales_Orders__r.size() == 0);
        System.assertNotEquals('A sales order has been created', o.Notes__c);
    }
}
```

Test Data Creation

Invalid Input

Test Method Execution

Test Result Verification

Permission-Based Tests

Permission-based tests ensure that **code behaves as expected** when it is **run as a user with or without specific permissions**.

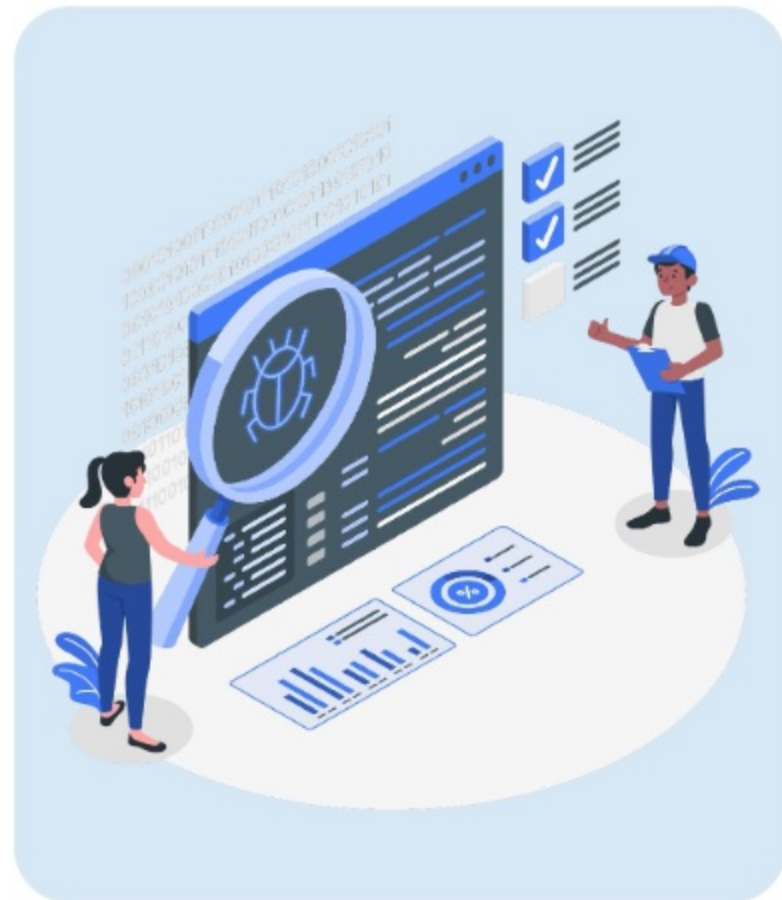
❖ IMPORTANCE

Permission-based testing verifies that an **application's access controls** work as expected, ensuring that users have **appropriate access to resources** based on their **permissions**.

❖ PATTERN

The pattern for writing positive unit tests uses the following steps:

- 1) Generate or load **test data**
- 2) Create **test users**
- 3) Execute test using **System.runAs(user)**
- 4) Combine **positive** and **negative test patterns**



Permission-Based Tests

The following example uses `System.runAs()` to run code in the **context of a specific user** to test code behavior and record access to the user.

```
@isTest
public static void test1() {

    Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User'];
    User newUser = new User(Email='newuser@acme.com',UserName='newuser@acme.com.dev',LastName='Testing',Alias='newUser',
    ProfileId=p.Id,LanguageLocaleKey='en_US',LocaleSidKey='en_US',TimeZoneSidKey='America/Los_Angeles',EmailEncodingKey='UTF-8');

    System.runAs(newUser) {
        System.assertEquals('newuser@acme.com.dev', UserInfo.getUserName());
        System.assertEquals(p.Id, UserInfo.getProfileId());
    }

    User currentUser = [SELECT Id, ProfileId FROM User WHERE UserName='currentuser@acme.com.dev'];
    System.runAs(currentUser) {
        System.assertNotEquals('newuser@acme.com.dev', UserInfo.getUserName());
        System.assertNotEquals(p.Id, UserInfo.getProfileId());
    }
}
```

Code execution as *newUser*

Code execution as *currentUser*

Mocks and Stubs

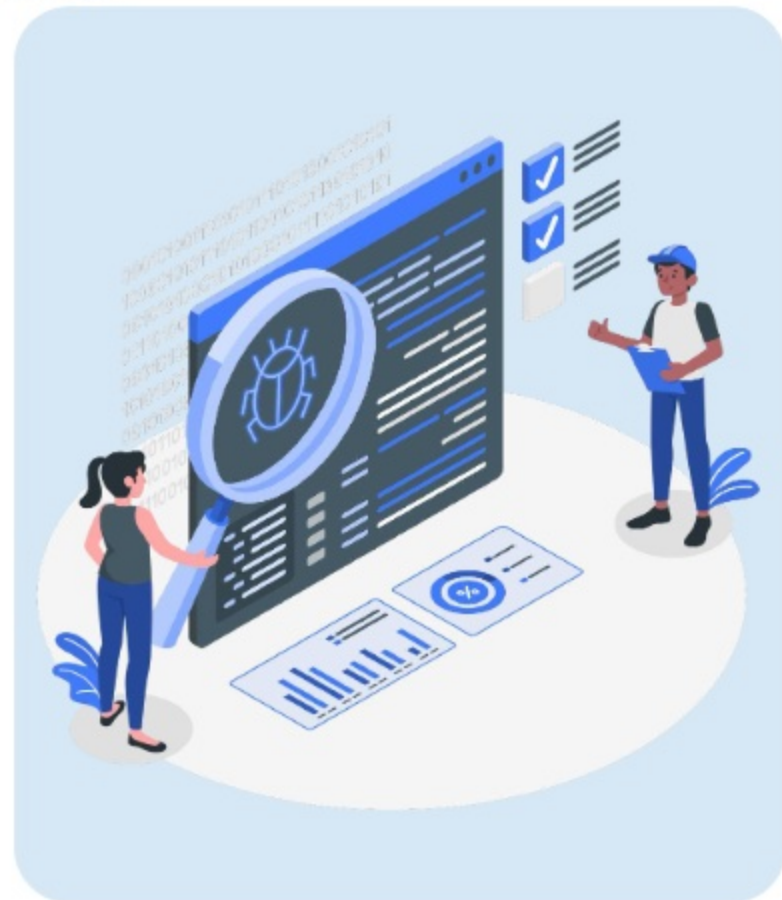
Mocks and stubs facilitate unit testing by **simulating real objects and methods**, allowing developers to test code in isolation from external dependencies.

❖ MOCKS & STUBS

A mock is used to **mimic the behavior of complex real objects** and is utilized at the **object level**. A stub provides **predefined responses to method calls**.

❖ USING MOCKS & STUBS

A mock is used to **test interactions with external systems**, such as HTTP callouts to third-party web services, by simulating responses. Stubs are used to **test code that relies on the internal state or behavior of other objects**, allowing the simulation of various conditions and responses without relying on the real implementation.



Mocks and Stubs

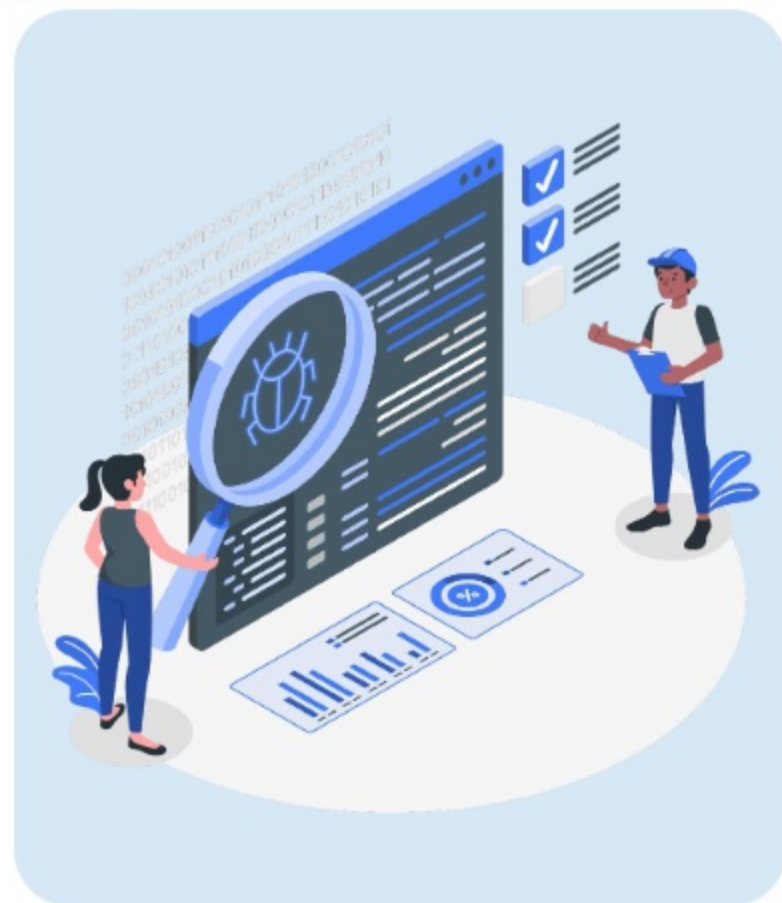
Mocks and stubs facilitate unit testing by **simulating real objects and methods**, allowing developers to test code in isolation from external dependencies.

❖ MOCK CALLOUTS

The **HttpCalloutMock** interface can be implemented to simulate HTTP callouts. The **Test.setMock()** method allows associating the mock with HTTP callouts.

❖ STUBBING

The **StubProvider** interface is used to replace the behavior of methods within the test context. It is useful to **test functionality that depends on the behavior of other classes or methods**.



HTTP Callout Mock

The following example shows the use of `HttpCalloutMock` to simulate an HTTP response.

```
// Apex class that implements the HttpCalloutMock interface to simulate the response of a successful enrollment of a student:
public class EnrollHttpCalloutMock implements HttpCalloutMock {
    public HttpResponse respond(HttpRequest req) {
        HttpResponse res = new HttpResponse();
        res.setHeader('Content-Type', 'application/json');
        res.setBody('{"success":true,"studentId":12345,"courseId":67,"yearLevel":3}'); // create fake response
        res.setStatusCode(200);
        return res;
    }
}

// Code contained and executed inside a test method:
Test.startTest();
Test.setMock(HttpCalloutMock.class, new EnrollHttpCalloutMock());
StudentApplicationWrapper application = generateStudentApplication(); // some helper method
Enrollment erm = StudentController.enroll(application); // invoke the method to test

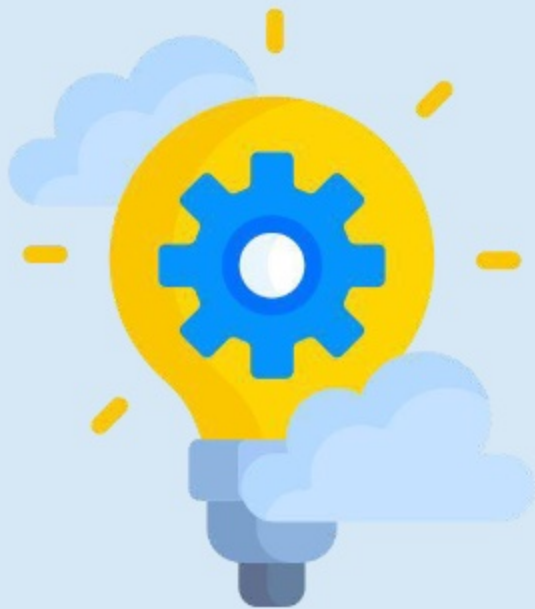
// verify new record created from fake data
System.assertEquals(erm.ExternalStudentId__c, 12345);
System.assertEquals(erm.YearLevel__c, 3);
System.assertEquals(erm.CourseId__c, 67);
Test.stopTest();
```

Business Scenario

Table of Contents 

Business Scenario

Read the business scenario below and determine the most appropriate solutions for the company's requirements.



Cosmic Cloud Solutions, a leading provider of cloud-based enterprise solutions, serves a wide array of industries by offering scalable software solutions that support business operations, ranging from supply chain management to customer relationship management (CRM) systems. The company prides itself on its agile development practices and commitment to delivering high-quality, robust software products. It utilizes a complex Salesforce environment to manage its sales processes, customer support, and internal operations.

The company's Salesforce development team has encountered issues with ensuring the quality and reliability of Apex code, particularly when dealing with diverse data scenarios. The company seeks the assistance of a Salesforce architect to recommend test data strategies and unit testing methods for various technical requirements.

Requirements & Solutions

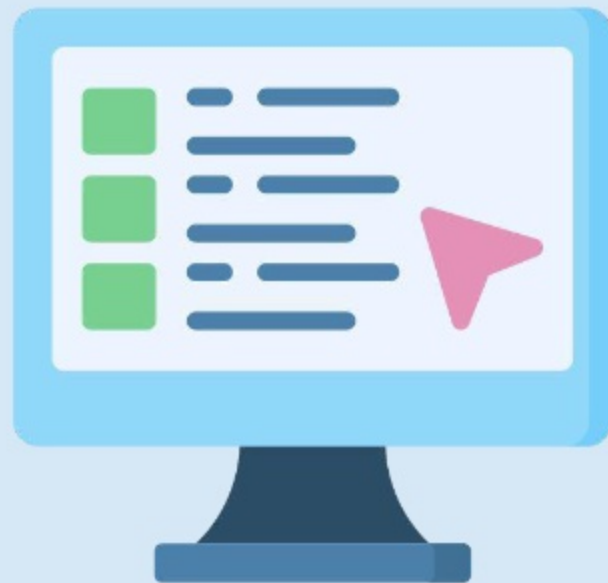
[Table of Contents](#) 

Requirement 1



SCENARIO

The company recently removed a record type and added a new record type for ERP solutions on the Opportunity object. As a result, certain unit tests in a test class that references the Opportunity object have been failing. The architect needs to recommend a suitable approach to ensure that tests execute predictably. Furthermore, a developer is working on a test class for a new functionality that processes a large volume of opportunity data. A suitable test data strategy is required for this use case.



Solution 1



SOLUTION

To ensure that unit tests in the test class execute predictably, the **test setup method** in the test class should be updated. The test class should **create its own test data** and **dynamically query for available record type Ids** rather than hard-coding them. This would ensure that tests remain resilient to changes in record types. For large data volume testing, a **CSV file** containing the required number of opportunity records can be uploaded as a **static resource**. It can then be referenced in test classes.



Requirement 2



SCENARIO

A team of senior developers in the IT department recently conducted a comprehensive code review of the support management application and found that Apex tests do not use enough negative unit tests. In addition, the existing test data strategy used for testing the inbound call volume functionality needs to be evaluated. The support department currently receives almost 300 calls per hour, but the existing test data being used in test classes is suitable for 150 calls per hour.

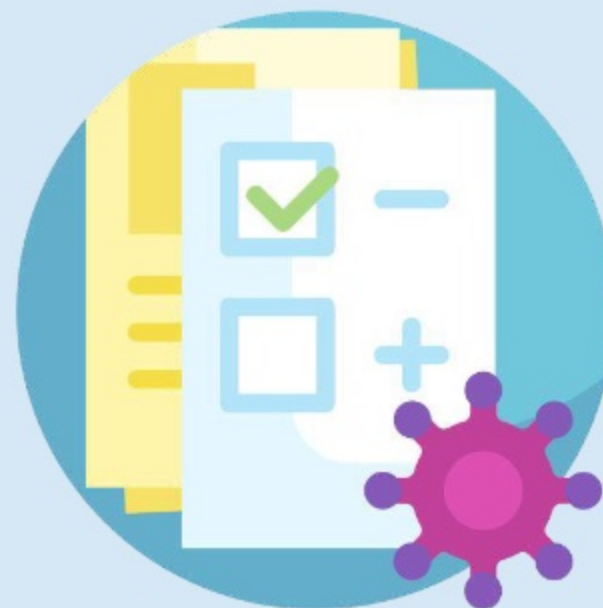


Solution 2



SOLUTION

To enhance negative unit testing, all the **critical paths and potential failure points** within the Apex classes should be identified. Thereafter, additional unit test methods should be created for **negative test cases**. **Permission-based testing** can be utilized to ensure that some negative tests cover scenarios where operations fail due to insufficient permissions or access rights, simulating different user roles interacting with the application. For the inbound call volume testing, the volume of test data should be increased by **100%** to more accurately reflect the current operational volume of 300 calls per hour. Test data generation scripts or methods should be updated accordingly.



Learn More

-  [Advanced Apex Testing](#)
-  [Understanding Test Data](#)
-  [Apex Testing](#)

